

What is Python

What is Python?

- Python is a **high-level, interpreted language**
- Used in **Web, ML, AI, Automation**
- Python reads code **top to bottom**

In simple Words : Python is a popular programming language used to tell computers what to do. It's known for being simple to read and write, which makes it a great choice for beginners and professionals alike.

Python is very versatile. People use it for:

- **Web development** (building websites and web apps)
 - **Data analysis & science** (working with data, statistics, AI)
 - **Automation** (writing scripts to do repetitive tasks)
 - **Game development**
 - **Machine learning & AI**
-

Why is Python popular?

- **Easy to learn:** Its syntax looks a lot like plain English
 - **Powerful:** Used by companies like Google, Netflix, and NASA
 - **Huge community:** Lots of tutorials, libraries, and support
 - **Cross-platform:** Works on Windows, macOS, and Linux
-

Example of Python code

Here's a simple Python program:

`print("Hello, world!")` → This will print Hello World! on the screen

Installing IDE (*Integrated Development Environment.*) :

What is an IDE?

An IDE is a software application that provides all the tools you need to write, run, and debug code in one place.

Key components of an IDE:

- **Code Editor** → where you write code
- **Compiler/Interpreter** → runs your code
- **Debugger** → helps find and fix errors
- **Auto-completion** → suggests code while typing

Examples of IDEs:

- PyCharm
- Visual Studio Code
- Jupyter Notebook

You can install anyone of them here is some reference video link to install

[Install Jupyter Notebbok for windows 11](#)

[How to Setup Python in Visual Studio Code on Windows 11](#)

And can refer multiple youtube videos for further any help

Tokens in Python

Tokens are the **smallest units (building blocks)** of a Python program. When you write Python code, the interpreter breaks it into tokens before execution.

Example :

```
x = 10 + 5
```

Here Tokens are:

- `x` → Identifier
- `=` → Operator
- `10` → Literal
- `+` → Operator
- `5` → Literal

Analogy : As If I say My name is XYZ so yha par we have 4 words so jaise english sentence m words eak smallest unit hua jaise ki yha par hmare pass 4 words hai koi noun hai koi verb hai and so on similarly python language m bhi eak line of code k chotr chote units ko token khte hai jaisa ki upar wle example m dikhya gya hai

Types of Tokens in Python

1. Identifiers (Names)

Identifiers are names given to variables, functions, classes, etc.

Examples:

```
name = "Sam"  
age = 21
```

Rules:

- Must start with a letter (a–z, A–Z) or `_`
 - Cannot start with a digit
 - Cannot use keywords
 - Case-sensitive (`age` ≠ `Age`)
-

2. Keywords (Reserved Words)

Keywords are special words with predefined meaning in Python.

Examples:

- `if, else, while, for, def, class, True, False, None`

Example:

```
if age > 18:  
    print("Adult")
```

Python provides these keywords officially via the Python Software Foundation.

3. Literals (Constant Values)

Literals are fixed values in a program.

Types of literals:

- **Numeric literals**

```
x = 10    # int  
y = 3.14  # float
```

- **String literals**

```
name = "Python"
```

- **Boolean literals**

```
is_valid = True
```

- **Special literal**

```
value = None
```

4. Operators

Operators are symbols used to perform operations on variables and values.

Types of operators:

- **Arithmetic Operators**

+ - * / % **

- **Relational Operators**

== != > < >= <=

- **Logical Operators**

and or not

- **Assignment Operators**

= += -= *= /=

Variables & Data Types

A variable is a **container to store data**

Coding

```
age = 20           / integer
price = 99.5       / float
name = "Sam"       / string
is_student = True  / boolean
```

```
print(age)
print(price)
print(name)
print(is_student)
```

Task : Give Output of each

Data Type	Example
int	10, -5
float	10.5, 3.14
str	"Hello"
bool	True, False

Common Mistakes to Highlight

```
is_student = true    -> Wrong  
is_student = True    -> Correct
```

type() Function

Why Needed?

To check **what type of data** a variable holds.

Coding

```
x = 10  
y = 10.5  
z = "Python"  
  
print(type(x))  
print(type(y))  
print(type(z))
```

Explain:

- Python **automatically decides** the type
 - This is called **Dynamic Typing**
-

Type Casting

Problem Statement

User input always comes as **string**

Coding

```
a = "10"  
b = "20"  
print(a + b)
```

Output: 1020

Fix Using Type Casting

```
a = int("10")  
b = int("20")  
print(a + b)
```

More Examples

```
x = float(5)  
y = int(3.7)  
z = str(100)  
  
print(x, y, z)
```

Students Task

Convert:

- "25" → int
 - 12 → float
 - False → int
-

print() Function & Formatting

Basic print

```
print("Hello Python")  
print(10)
```

Multiple Values

```
name = "Rahul"  
age = 21
```

```
print(name, age)
```

Formatted Output (Very Basic)

```
print("My name is", name, "and age is", age)
```

Output : My name is Rahul and age is 21

Explanation :

Aisa iis liye hua kyuki hm print function m double quotes k inside jo bhi likha hota hai wo as it is print hota hai screen mai and jo outside (" ") m hota hai uska jo value store hota hai wo value print hota h screen p

Jaisa ki yha name and age mai maine value store kiya which is outside the quotes thus value is printed for those and remaining are inside quotes thats why wo as it is print hua hai yha par.

Till now we had seen what is **Python** then **Tokens in Python** then about **type function** then about **typecasting** and about **print function** which is used to display the output on screen

Taking input from the user

```
Age = input( " Enter the value of age : " )
```

So basically to take input from user we use **Input()** function

Yha par bhi jo quotes k under hai wo as it is print hoga screen pe and then we have to input the value

Lekin hmm jo bhi input dege wo string k form jayega so type casting is important
Agar hme input m age ki value chaiye to kya hoga code

```
Age = int (input( " Enter the value of age : " ) )
```

Aab hmara age ka value integer m hoga

Comments in Python

What are Comments?

Comments are lines in a program that are **ignored by the Python interpreter**.

They are used to **explain code**, improve readability, and make maintenance easier.

Mtlb comment wo line of code hai jo run compiler nhi hoga hmm bas isko program m likthe hai taki hmm code of self explanatory bana sake taki koi dosra bhi hmara code ko dekh k and comments ko padh kar code smjh sake

Types of Comments in Python

1. Single-Line Comments

Created using **#**

Code Example:

```
# This is a single-line comment  
print("Hello World")
```

Here First line is ignored because it is a comment and second line will run

2. Multi-Line Comments

Python doesn't have a special symbol for multi-line comments, but we use:

Method 1: Multiple **#**

```
# This is a comment  
# written in multiple lines  
# using hash symbol
```

Method 2: Triple Quotes (' ' ' or " " ")

```
"""
This is a multi-line comment
or docstring
"""
print("Python")
```

Docstrings

Docstrings are special comments used to describe functions, classes, or modules.

Example:

```
def add(a, b):
    """ This function returns sum of two numbers """
    return a + b
```

Used for documentation and can be accessed using:

```
print(add.__doc__)
```

Basically docstring function ko explain krne m use hota ye bhi eak trh k comment hi hai jo ki function k baare m batane k liye use hota h ki iska output kya hoga ye kya return krega as so on.

Rules & Best Practices

- Keep comments **short and meaningful**
- Don't write obvious comments

Example x = 5 # assigning 5 to x (not useful)

Write helpful comments

```
# Store user age for eligibility check
age = 18
```

Note: Triple quotes are actually strings, but often used as comments when not assigned.

See here we assigned it to a variable this acting as a string

main.py	Run	Output
<pre>1 # Assign a string using triple quotes 2 name = """My name is Shekhar Kumar """ 3 print(name) 4 # Printing a empty line 5 print() 6 details = "Currently I am working as a trainer in SDBCE" 7 print(details) 8 # Printing a empty line 9 print() 10 Age_D = '''I am x years old''' 11 print(Age_D) 12 # Printing a empty line 13 print() 14 address = 'I live near Indore' 15 print(address)</pre>		<pre>My name is Shekhar Kumar Currently I am working as a trainer in SDBCE I am x years old I live near Indore === Code Execution Successful ===</pre>

Operators in Python

Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example
+	Addition	<code>a + b</code>
-	Subtraction	<code>a - b</code>
*	Multiplication	<code>a * b</code>
/	Division	<code>a / b</code>
%	Modulus (remainder)	<code>a % b</code>
**	Exponent (power)	<code>a ** b</code>
//	Floor division	<code>a // b</code>

main.py	   Share	Run	Output
<pre>1 a = 20 2 b = 3 3 print(a+b) 4 print(a-b) 5 print(a*b) 6 print(a/b) 7 8 # Floor division gives only integer value removing the decimal values which is small than actual division value 9 print(a//b) 10 11 # % gives the remainder when a(10) is divided by b(3) 12 print(a%b) 13 14 # ** gives a to power b here 10 ka power 3 = 1000 15 print(a**b)</pre>			<pre>23 17 60 6.666666666666667 6 2 8000 === Code Execution Successful ===</pre>

2. Comparison (Relational) Operators

Used to compare two values.

Result is **True** or **False**.

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater or equal
<code><=</code>	Less or equal

Example:

```
x = 10
y = 5
print(x > y) # True
print(x == y) # False
```

Student Task : Give Output for this :

```
X = 11
Y = 11
print(X >= Y)
print(X <= Y)
```

3. Logical Operators

Used to combine conditions.

Operator	Meaning
and	True if both are true
or	True if at least one is true
not	Reverses condition

Example:

```
a = True
b = False
print(a and b) # False
print(a or b)  # True
print(not a)   # False
```

4. Assignment Operators

Used to assign values to variables.

Operator	Example	Meaning
=	x = 5	Assign
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3

Example:

```
x = 5
x += 2
print(x) # 7
```

5. Bitwise Operators

Work on binary (bits) representation of numbers.

Operator	Meaning
&	AND
	OR
^	XOR
~	NOT
<<	Left shift
>>	Right shift

Example:

```
a = 5 # 101
b = 3 # 011
print(a & b) # 1
print(a | b) # 7
```



6. Membership Operators

Used to check if a value exists in a sequence.

Operator	Meaning
----------	---------

<code>in</code>	Present
-----------------	---------

<code>not in</code>	Not present
---------------------	-------------

Example:

```
name = "python"  
print('p' in name)    # True  
print('z' not in name) # True
```

7. Identity Operators

Used to compare memory locations of objects.

Operator	Meaning
----------	---------

<code>is</code>	Same object
-----------------	-------------

<code>is not</code>	Different object
---------------------	------------------

Example:

```
a = [1, 2]  
b = a  
print(a is b) # True
```

Operator Precedence

Order in which operations are performed:

1. `**` (Exponent)
2. `*` `/` `//` `%`
3. `+` `-`
4. Comparison operators
5. Logical operators (`not`, `and`, `or`)

Example:

`print(2 + 3 * 4) # 14 (not 20)`

If-else in Python

if-else is used for decision making in Python. It allows the program to execute different blocks of code based on conditions.

Simple Example

```
age = 18

if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

Important Rule :

Indentation is mandatory

Below example is wrong because there is no tab separation (indentation)

```
if age >= 18:
print("Eligible")
```

Here below example is correct

```
if age >= 18:
    print("Eligible")
```

Student Practice : Give Output

```
marks = 40
if marks >= 35:
    print("Pass")
else:
    print("Fail")
```

if-elif-else Ladder

Used when there are multiple conditions.

```
marks = 75
```

```
if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Nested if

An `if` inside another `if`.

```
num = 10
```

```
if num > 0:
    if num % 2 == 0:
        print("Positive Even Number")
```

Quick Recap + Mini Quiz

1. Data type of `"10"`?
2. Output of `int("5") + 2`?
3. Output of: `print(10 > 3 and 5 < 2)`

Homework

- Write a program:
 - Store name & age and Print eligible/not eligible for vote

Loops in Python ?

Why Loops?

Problem:

```
print("Hello")  
print("Hello")  
print("Hello")  
print("Hello")  
print("Hello")
```

What if I want to print this 100 times?

Loops repeat work automatically

for Loop

Used when:

We know **how many times** you want to run the loop or we have to iterate over a sequence

Syntax:

```
for variable in sequence:  
    # code block
```

Example 1: Print numbers 1 to 5



Python Online Compiler

main.py	Run	Output
<pre>1 for i in range(1, 6): 2 print(i) 3</pre>		1 2 3 4 5

Example 2: Loop through a string

main.py	Run	Output
<pre>1 for ch in "Python": 2 print(ch) 3</pre>		P y t h o n === Code Execution Successful ===

Example 3: Loop through a list

main.py	Run	Output
<pre>1 numbers = [10, 20, 30] 2 for num in numbers: 3 print(num) 4</pre>		10 20 30 === Code Execution Successful ===

range()

Used to generate a sequence of numbers.

range(start, stop, step)

Example:

main.py	   Share	Run	Output
1 ▾ for i in range(0, 10, 2):			0
2 print(i)			2
3			4
			6
			8
			=== Code Execution Successful ===

while Loop

Used when:

You don't know how many times the loop will run but we know the stopping condition

♦ Syntax:

```
while condition:  
    # code block
```

Example:

main.py	Run	Output
<pre>1 i = 1 2 while i <= 5: 3 print(i) 4 i += 1 5</pre>		<pre>1 2 3 4 5 === Code Execution Successful ===</pre>

Infinite Loop (Important)

```
while True:  
    print("Hello")
```

Runs forever unless stopped manually

Loop Control Statements

1. break

Stop the loop completely.

main.py	Run	Output
<pre>1 for i in range(1, 6): 2 if i == 3: 3 break 4 print(i) 5</pre>		<pre>1 2 === Code Execution Successful ===</pre>

2. continue

Skips current iteration.

main.py	Run	Output
<pre>1 for i in range(1, 6): 2 if i == 3: 3 continue 4 print(i) 5</pre>		<pre>1 2 4 5 === Code Execution Successful ===</pre>

3. pass

Does nothing (placeholder).

```
for i in range(5):  
    Pass
```

Pass likhne se hme error nhi milega code ko run krne p man lo abhi hme function k logic nhi pta we want to write it later but hme use krna hai apne code m uss function ko then there also we just declare the function and write pass inside it to avoid error

Nested Loops

Loop inside another loop.

Example:

```
1 for i in range(1, 4):  
2     for j in range(1, 3):  
3         print(i, j)  
4
```

1 1
1 2
2 1
2 2
3 1
3 2

=== Code Execution Successful ===

else with Loops (Important for Exams)

Python allows `else` with loops.

```
for i in range(3):  
    print(i)  
else:  
    print("Loop finished")
```

`else` runs when loop ends normally (not by `break`)

main.py	   Share 	Output
<pre>1 ▾ for i in range(3): 2 print(i) 3 ▾ else: 4 print("Loop finished") 5</pre>	<pre>0 1 2 Loop finished</pre>	

Sequence Datatypes in Python

List in Python

In **Python**, a **list** is a built-in data structure used to store multiple items in a single variable.

It is **ordered, mutable, and allows duplicate values**.

Creating a List

```
L = [] # Empty List  
my_list = [1, 2, 3, 4, 5]  
print(my_list)
```

♦ Different Types of Lists

Integer list

```
numbers = [10, 20, 30]
```

String list

```
names = ["Ram", "Shyam", "Mohan"]
```

Mixed list

```
mixed = [1, "Hello", 3.5, True]
```

◆ Accessing Elements

```
my_list = [10, 20, 30, 40]
```

```
# Indexing
```

```
print(my_list[0]) # First element
```

```
print(my_list[-1]) # Last element
```

◆ List Slicing

main.py	   Share	Run	Output
1 my_list = [1, 2, 3, 4, 5]			[2, 3, 4]
2 print(my_list[1:4])			[5, 4, 3, 2, 1]
3			
4 my_list = [1, 2, 3, 4, 5]			=== Code Execution Successful ===
5 print(my_list[: : -1]) # reverses the list			
6			

◆ Common List Methods

main.py		Output
1 my_list = [1, 2, 3]		[1, 2, 3]
2 print(my_list)		[1, 2, 3, 4]
3 my_list.append(4) # Add element		[1, 10, 2, 3, 4]
4 print(my_list)		[1, 10, 3, 4]
5 my_list.insert(1, 10) # Insert at index		[1, 10, 3]
6 print(my_list)		[1, 3, 10]
7 my_list.remove(2) # Remove element		[10, 3, 1]
8 print(my_list)		
9 my_list.pop() # Remove last		=== Code Execution Successful ===
10 print(my_list)		
11 my_list.sort() # Sort list		
12 print(my_list)		
13 my_list.reverse() # Reverse list		
14 print(my_list)		
15		

```
my_list = [10, 20, 30, 40]
```

```
del my_list[1] # removes element at index 1
```

```
print(my_list) # [10, 30, 40]
```

```
del my_list[1:3] # removes slice
```

```
print(my_list) # [10]
```

```
del my_list # deletes entire list
```

```
# print(my_list) → Error (list no longer exists)
```


`clear()` removes **all elements**, but the list **still exists**.

Example:

```
my_list = [10, 20, 30]
```

```
my_list.clear()
```

```
print(my_list) # []
```

Tuples in Python

What is a Tuple?

A tuple is a built-in data structure in Python that stores multiple items in a single variable.

Key idea: Tuple = ordered + immutable collection

```
t = (1, 2, 3)
```

Properties of Tuple

Property	Description
Ordered	Elements maintain their order
Immutable	Cannot be changed after creation
Indexed	Access using index
Allows duplicates	Same values allowed
Heterogeneous	Can store different data types

Example : `t = (1, "hello", 3.5, True)`

Creating Tuples

Method 1: Using parentheses

```
t = (1, 2, 3)
```

Method 2: Without parentheses

```
t = 1, 2, 3
```

Single element tuple (important)

```
t = (5,) # comma is required
```

```
T = 5 ,
```

Empty tuple

```
t = ()
```

Accessing Elements

Indexing

```
t = (10, 20, 30)
```

```
print(t[0]) # 10
```

Negative indexing

```
print(t[-1]) # 30
```

Slicing

```
print(t[0:2]) # (10, 20)
```

Looping Through Tuple

```
t = (10, 20, 30)
```

```
for i in t:
```

```
    print(i)
```

Nested Tuples

```
t = (1, (2, 3), (4, 5))
```

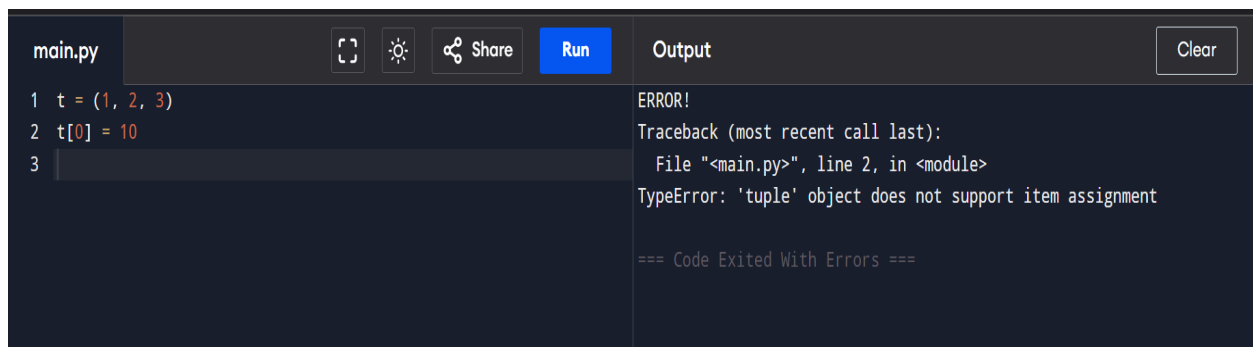
```
print(t[1][0]) # 2
```

Tuple is Immutable

```
t = (1, 2, 3)
```

```
t[0] = 10
```

this will give error



```
main.py  [ ] [ ] [ ] Share Run Output Clear
1 t = (1, 2, 3)
2 t[0] = 10
3
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 2, in <module>
TypeError: 'tuple' object does not support item assignment

=== Code Exited With Errors ===
```

But if tuple contains mutable objects:

```
t = (1, [2, 3])
```

```
t[1][0] = 100 # allowed
```

`print(t)` → **Output : (1, [100, 3])**

main.py	Output
<pre>1 t = (1, [2, 3]) 2 print(t) 3 print() 4 t[1][0] = 100 # allowed 5 print(t) 6</pre>	<pre>(1, [2, 3]) (1, [100, 3]) === Code Execution Successful ===</pre>

Tuple Packing & Unpacking

Packing

main.py	Output
<pre>1 t = 1, 2, 3 2 print(t)</pre>	<pre>(1, 2, 3) === Code Execution Successful ===</pre>

Unpacking

main.py	Output
<pre>1 a, b, c = (1, 2, 3) 2 3 print(a) 4 print(b) 5 print(c)</pre>	<pre>1 2 3 === Code Execution Successful ===</pre>

Extended unpacking

main.py	   Share  Run	Output 
<pre>1 a, *b = (1, 2, 3, 4) 2 print(a) # 1 3 print(b) # [2, 3, 4] 4</pre>		<pre>1 [2, 3, 4] === Code Execution Successful ===</pre>

Tuple Operations

Concatenation

```
t1 = (1, 2)
t2 = (3, 4)
print(t1 + t2)
```

Repetition

```
print((1, 2) * 3)
```

Membership

```
print(2 in (1, 2, 3))
```

Built-in Functions for Tuple

```
t = (1, 2, 3, 2, 2)
```

```
print(len(t))  # length
```

```
print(max(t))  # max value
```

```
print(min(t))  # min value
```

```
print(sum(t))  # sum
```

Tuple Methods

Only 2 methods exist:

```
t = (1, 2, 2, 3)
```

```
print(t.count(2))  # 2
```

```
print(t.index(3))  # 3
```

Why is tuple faster?

Because it is immutable, Python optimizes it.

SET in Python

What is a Set?

- Unordered collection
- No duplicate elements
- Mutable (but elements must be immutable)

Creating a Set

`S = set()` # for empty set

`S = { 1 }` # for singleton set

`s = {1, 2, 3}` # for non empty set

Core Operations

1. Add / Remove

```
s = {1,2,3}
s.add(10)
s.update([4,6])
print(s)
```

```
s.remove(2)    # error if not present
s.discard(5)   # no error
s.pop()        # removes random element
s.clear()
del s
```

2. Set Operations

A = {1,2,3}

B = {3,4,5}

```
print(A | B) # Union → {1,2,3,4,5}
print(A & B) # Intersection → {3}
print(A - B) # Difference → {1,2}
print(A ^ B) # Symmetric Difference → {1,2,4,5}
```

Practice Patterns

1. Remove duplicates

```
arr = [1,2,2,3,4,4]
unique = list(set(arr))
```

2. Find common elements

```
a = [1,2,3]
b = [2,3,4]

common = list(set(a) & set(b))
```

3. Check subset

```
A = {1,2}
B = {1,2,3}

print(A.issubset(B)) # True
```

4. Count distinct elements

```
arr = [1,2,2,3,4]
print(len(set(arr)))
```

Write Python code to find First repeating element in a list

```
arr = [1,2,3,2,1]
seen = set()

for x in arr:
    if x in seen:
        print(x)
        break
    seen.add(x)
```

DICTIONARY in Python

What is a Dictionary?

- Key-value pair
- Keys must be unique & immutable
- Value can be duplicate and immutable
- Dictionary is immutable

Creating a dictionary

```
d = { } # for empty dictionary
d = dict ()
```

```
d = dict ( name = "Sam" , age = 22 )
```

```
d = {"name": "Sam", "age": 22}
```

Core Operations

Access values from dictionary

```
d = {"name": "Sam", "age": 22}
print(d["name"])
print(d.get("age")) # safe

print( d.keys() )
print(d.values() )
print(d.items() )
```

Add / Update

```
d = {"name": "Sam", "age": 22}
d["city"] = "Delhi"
d["age"] = 23

print(d)
```

Delete

```
del d["age"]
d.pop("name")

d.clear()
del d
```

Looping

```
for key in d:
    print(key, d[key])

for k, v in d.items():
    print(k, v)
```

Most Important Patterns

♦ 1. Frequency Count

```
arr = [1,2,2,3,3,3]
```

```
freq = {}
```

```
for x in arr:
```

```
    freq[x] = freq.get(x, 0) + 1
```

```
print(freq)
```

♦ 2. Using Counter (Shortcut)

```
from collections import Counter
```

```
arr = [1,2,2,3,3,3]
```

```
freq = Counter(arr)
```

```
print(freq)
```

♦ 3. Find first non-repeating element

```
arr = [1,2,2,3,3,4]
```

```
freq = {}
```

```
for x in arr:
```

```
    freq[x] = freq.get(x, 0) + 1
```

```
for x in arr:
```

```
    if freq[x] == 1:
```

```
        print(x)
```

```
        break
```

Strings in Python

Note for strings in Python

What is a Function

What is a Function?

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code again and again, we create a function and call it whenever needed.

Why Functions are Important

Functions help in:

- Code reusability
- Reducing code duplication
- Better readability
- Easier debugging
- Easier maintenance
- Dividing large programs into smaller modules

Basic Syntax of Function

```
def function_name(parameters):
```

```
    # function body
```

```
    statements
```

```
    return value
```

Example

```
def greet():  
    print("Hello AIML Students!")
```

Function Call:

```
greet()
```

Important Terminologies

Term	Meaning
Function Definition	Creating function
Function Call	Using function
Parameters	Variables in function definition
Arguments	Actual values passed
Return Value	Value sent back

Types of Functions

Functions are mainly of two types:

1. Built-in Functions
2. User-defined Functions

User Defined Functions

Functions created by programmer are called **user-defined functions**.

We use **def** keyword.

Syntax

```
def function_name():
```

```
    statements
```


Function Without Parameters

```
def message():  
    print("Good Morning")
```

Calling:

```
message()
```

Function With Parameters

Types of based on Arguments & Return

(a) Without return

```
def add(a, b):  
    print(a + b)
```

```
add(2, 3)
```

(b) With return

```
def add(a, b):  
    return a + b
```

```
result = add(2, 3)  
print(result)
```

Important:

- **return** → sends value back
 - **print** → just displays
-

Return Multiple Values

```
def calc(a, b):  
    return a+b, a-b, a*b  
  
x, y, z = calc(10, 5)  
  
print(x,y,z)
```

Types of Arguments

1. Positional Arguments

```
def student(name, age):  
    print(name, age)
```

```
student("Sam", 22)
```

```
def student(name, age):  
    print(name)  
    print(age)
```

```
student(20, "Aman")
```

This one is Wrong but upar wla correct hai as jis order m parameter defined hai ussi order m arguments ja rhe h

2. Keyword Arguments

```
def student(name, age):  
    print(name, age)
```

```
student(age=22, name="Sam")
```

Mixing Positional and Keyword Arguments

Allowed, but positional must come first.

```
def demo(a, b, c):  
    print(a, b, c)  
demo(10, c=30, b=20)
```

demo(a=10, 20, c=30) → Wrong

3. Default Arguments

Parameters having default values.

If the value not passed, default is used.

```
def greet(name="Guest"):  
    print("Hello", name)
```

Function Call :

```
greet()          # Without passing the parameter  
greet("Sam")     # with parameter passed
```

4. Variable Length Arguments

Sometimes number of arguments is unknown.

Python provides:

- `*args`
 - `**kwargs`
-

***args (Variable Positional Arguments)**

Allows multiple positional arguments.

Stored as tuple.

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1,2,3,4))  
print(total(1,2,3))
```

****kwargs (Variable Keyword Arguments)**

Allows multiple keyword arguments.

Stored as dictionary.

```
def details(**info):  
    print(info)  
  
details(name="Sam", age=22)
```

What is Anonymous Function?

Anonymous means: A function without a name.

In Python, anonymous functions are created using: **lambda**

That is why anonymous functions are also called: **Lambda Functions**

Lambda Functions (One-line functions)

Syntax of Lambda Function

lambda arguments : expression

Example

```
square = lambda x: x * x  
print(square(5))
```

Equivalent normal function:

```
def square(x):  
    return x * x
```

Why Use Lambda Functions?

Lambda functions are useful when:

- Function is small
 - Function is needed temporarily
 - Function is used only once
 - Passing function as argument
-

Lambda with Multiple Arguments

```
multiply = lambda a, b: a * b  
print(multiply(4, 5))
```

Lambda Without Variable

You can directly call lambda.

```
print((lambda a, b: a + b)(10, 20))
```

Lambda with Conditional Statements

```
largest = lambda a, b: a if a > b else b  
print(largest(10, 20))
```

Note : If function is small and temporary → use lambda.

Even/Odd Check

```
even = lambda x: x % 2 == 0
```

```
print(even(8))
```

```
print(even(5))
```

Higher Order Functions

Functions that:

- Take another function as argument or
- Return another function

Examples:

- `map()`
- `filter()`
- `reduce()`

`map()` Function

`map()` applies a function to every element of an iterable.

Syntax : `map(function, iterable)`

Flow of map()

List Elements



Function Applied One-by-One



New Modified Output

Example of map()

Square All Numbers

```
numbers = [1, 2, 3, 4, 5]
```

```
result = map(lambda x: x*x, numbers)
```

```
print(list(result))
```

Important Point

`map()` returns a map object.

Convert it using:

- `list()`
- `tuple()`
- `set()`

map() Using Normal Function

```
def square(x):  
    return x*x  
  
numbers = [1,2,3,4]  
  
result = map(square, numbers)  
  
print(list(result))
```

map() with Multiple Iterables

```
a = [1,2,3]  
b = [4,5,6]  
  
result = map(lambda x,y: x+y, a, b)  
  
print(list(result))
```

Example : Convert Names to Uppercase

```
names = ["rahul", "aman", "priya"]  
  
result = map(str.upper, names)  
  
print(list(result))
```

Example : Length of Strings

```
words = ["apple", "banana", "kiwi"]
```

```
result = map(len, words)
```

```
print(list(result))
```

filter() Function

`filter()` selects only those elements that satisfy a condition.

Syntax : `filter(function, iterable)`

Function must return: `True` or `False`

Flow of filter()

```
graph TD
    A[Input Elements] --> B[Condition Check]
    B --> C[True → Keep]
    B --> D[False → Remove]
```

Input Elements
↓
Condition Check
↓
True → Keep
False → Remove

Example of filter()

Example 1: Even Numbers

```
numbers = [1,2,3,4,5,6,7,8]
result = filter(lambda x: x % 2 == 0, numbers)

print(list(result))
```

filter() Using Normal Function

```
def even(x):
    return x % 2 == 0

numbers = [1,2,3,4,5]
result = filter(even, numbers)

print(list(result))
```

Important Note : Similar to map function filter function also create a filter object so convert it into list tuple or set

Example 1: Students Passed

```
marks = [35, 80, 20, 90, 50]

passed = filter(lambda x: x >= 40, marks)

print(list(passed))
```

Example 2: Filter Positive Numbers

```
numbers = [-2, 5, -7, 8, 10]

positive = filter(lambda x: x > 0, numbers)

print(list(positive))
```

Example 3: Filter Long Words

```
words = ["apple", "kiwi", "banana", "cat"]

result = filter(lambda x: len(x) > 4, words)

print(list(result))
```

Combining map() and filter()

```
numbers = [1,2,3,4,5,6,7,8]

even = filter(lambda x: x % 2 == 0, numbers)

squares = map(lambda x: x*x, even)

print(list(squares))
```

Assignment Questions

1. Create lambda for addition.
2. Find square using lambda.
3. Use map to double numbers.
4. Use filter to get positive numbers.
5. Sort tuples using lambda.
6. Filter prime numbers.

What is Function Aliasing?

Definition

Function aliasing means:

Giving another name (reference) to an existing function.

Just like a person can have multiple nicknames, a function can also have multiple names.

Why Function Aliasing Exists?

Because in Python:

- Functions are treated as objects
 - Functions can be assigned to variables
 - Variables can store function references
-

Basic Example

```
def wish():  
    print("Good Morning")
```

```
greeting = wish
```

```
wish()  
greeting()
```

- Here Both names point to the SAME function object.

```
wish -----> Function Object  
greeting ----^
```

```
def add(a, b):  
    return a + b
```

```
sum1 = add
```

```
print(id(add))  
print(id(sum1))
```

Both IDs are the same.

Even if the original function name is deleted, alias still works.

Important Point

`sum1 = add` This is Correct.

But: `sum1 = add()` Wrong if parameters are missing.

Because:

- `add` → reference to function
 - `add()` → calling the function
-

Functions are First-Class Objects

Python treats functions like normal objects.

Functions can:

- Be assigned to variables
 - Passed as arguments
 - Returned from functions
 - Stored in collections
-

Example: Function Stored in Variable

```
def square(x):  
    return x * x
```

```
f = square
```

```
print(f(5))
```

Example: Function Passed as Argument

```
def display(fun, value):  
    print(fun(value))
```

```
def cube(x):  
    return x ** 3
```

```
display(cube, 3)
```

Example: Returning Function

```
def outer():  
    def inner():  
        print("Inner Function")
```

```
    return inner
```

```
x = outer()
```

```
x()
```

Practical Uses of Function Aliasing

1. Shorter Names

```
import math
sq = math.sqrt

print(sq(25))
```

2. Callback Functions

Used heavily in:

- GUI programming
- Event handling
- Web development

Example:

```
def button_click():
    print("Button clicked")

callback = button_click

callback()
```

What will be Output of given Program

```
def multiply(a, b):
    return a * b
```

```
m = multiply
print(m(4, 5))
```

```
def show():
    print("Original")
```

```
x = show
del show
x()
```

Reduce Function

The `reduce()` function is used to: reduce a sequence into a single value.

It repeatedly applies a function on elements.

`reduce()` is inside **functools**

So to use it firstly we need to import it .

```
from functools import reduce
```

Syntax : `reduce(function, sequence)`

Parameter	Meaning
function	Function with 2 arguments this is important / mandatory
sequence	List/tuple/etc

Example

```
from functools import reduce
```

```
lst = [1, 2, 3, 4]
```

```
result = reduce(lambda x, y: x + y, lst)
```

```
print(result)
```

Using Normal Function

```
from functools import reduce
```

```
def add(x, y):  
    return x + y
```

```
lst = [10, 20, 30]
```

```
result = reduce(add, lst)
```

```
print(result)
```

Reduce with Multiplication

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(lambda x, y: x * y, numbers)
```

```
print(result)
```

Finding Maximum Element

```
from functools import reduce
```

```
lst = [10, 40, 5, 90, 30]
```

```
maximum = reduce(lambda x, y: x if x > y else y, lst)
```

```
print(maximum)
```

Reduce with Initial Value

Syntax: `reduce(function, sequence, initial)`

Example:

```
from functools import reduce
```

```
lst = [1, 2, 3]
```

```
result = reduce(lambda x, y: x + y, lst, 10)
```

```
print(result)
```

Output: 16

Initial value = 10

$10 + 1 = 11$

$11 + 2 = 13$

$13 + 3 = 16$

Reduce vs Map vs Filter

Function	Purpose
<code>map()</code>	Transform elements
<code>filter()</code>	Filter elements
<code>reduce()</code>	Produce single value

Important Internal Working

Reduce processes elements LEFT to RIGHT.

Example: `reduce(lambda x,y:x-y,[1,2,3])`

Working:

$1 - 2 = -1$

$-1 - 3 = -4$

Output: `-4`

Reduce with Strings

```
from functools import reduce
```

```
words = ["Python", "is", "awesome"]
```

```
sentence = reduce(lambda x, y: x + " " + y, words)  
print(sentence)
```

Advantages of Reduce

- Compact code
 - Functional programming style
 - Useful for aggregation
 - Cleaner mathematical operations
-

Disadvantages

- Sometimes less readable
 - Beginners may find confusing
 - Loops may be easier sometimes
-

Frequently Asked Interview Questions

Q1. Why is reduced inside functools?

Because Python 3 moved it from built-ins to functools module.

Q2. Does reduce return list?

No. It returns a SINGLE value.

Q3. Can reduce work on an empty list?

Without initial value → Error.

With initial value → Works.

Example:

```
reduce(lambda x,y:x+y, [], 0)
```

Output:

0

Important Error

```
reduce(lambda x,y:x+y, [])
```

Error:

TypeError

Because no starting value exists.

What is a Nested Function?

A function inside another function is called a nested function.

Example :

```
def outer():  
  
    print("Outer Function Started")  
  
    def inner():  
        print("Inner Function Executed")  
  
    inner()  
  
    print("Outer Function Ended")  
  
outer()
```

Important Point

Inner function cannot be called directly from outside.

```
def outer():  
  
    def inner():  
        print("Hello")  
    inner()  
outer()  
inner()
```

Output : NameError

Because `inner()` only exists inside `outer()`.

Why Use Nested Functions?

Nested functions help:

- Organize code
- Hide functionality
- Improve readability
- Avoid repetition
- Create closures
- Improve security

Example: Password Validation

```
def login_system():  
    def validate_password(password):  
        if len(password) >= 8:  
            return True  
        return False  
  
    password = input("Enter password: ")  
  
    if validate_password(password):  
        print("Valid Password")  
  
    else:  
        print("Invalid Password")  
  
login_system()
```

MINI PROJECT — STUDENT RESULT SYSTEM

```
def student_result():

    marks = []

    def add_marks():
        n = int(input("How many subjects? "))

        for i in range(n):
            m = int(input("Enter marks: "))
            marks.append(m)

    def calculate():
        total = sum(marks)
        percentage = total / len(marks)

        print("Total:", total)
        print("Percentage:", percentage)

    add_marks()
    calculate()

student_result()
```

Practice Problems

1. Function to check even/odd
2. Function to find maximum of 3 numbers
3. Function to reverse a string
4. Function to check palindrome

Hints :

```
def is_palindrome(s):
    return s == s[::-1]
```

5. Function to count vowels
6. Function to find factorial (iterative + recursive)
7. Prime number checker
8. Function to remove duplicates from list
9. Function to flatten nested list

Modules and Package

What is a Module?

A **module** is a Python file (`.py`) that contains Python code such as variables, functions, classes, and statements that can be reused in other programs.

Why Use Modules?

- Code reusability
- Better organization of code
- Easier maintenance
- Avoid writing the same code repeatedly

Importing Modules

1. Import Entire Module

```
import math
```

```
print(math.sqrt(25))
```

Output:

```
5.0
```

2. Import Specific Functions

```
from math import sqrt, factorial
```

```
print(sqrt(16))
```

```
print(factorial(5))
```

Output:

```
4.0  
120
```

3. Import with Alias

```
import numpy as np
```

```
arr = np.array([1, 2, 3])  
print(arr)
```

Output:

```
[1 2 3]
```

4. Import All Functions

```
from math import *
```

```
print(sqrt(49))
```

Output:

```
7.0
```

Note: Generally not recommended because it may create naming conflicts.

Types of Modules

1. Built-in Modules

Provided by Python itself.

Examples:

- math
- random
- datetime
- os
- sys

Example

```
import random
```

```
print(random.randint(1, 10))
```

2. User-Defined Modules

Created by programmers.

Example:

```
# mymodulename.py
```

```
name = "Sam"
```

```
def greet():  
    print("Welcome to Python")
```

Using the module:

```
import mymodulename
```

```
print(mymodule.name)  
mymodule.greet()
```

Common Built-in Modules

Math Module

```
import math
```

```
print(math.pi)  
print(math.sqrt(64))  
print(math.factorial(5))
```

Output:

```
3.141592653589793  
8.0  
120
```

Random Module

```
import random

print(random.randint(1, 100))
print(random.choice(['A', 'B', 'C']))
```

Datetime Module

```
import datetime

today = datetime.date.today()
print(today)
```

OS Module

Used for interacting with the operating system.

```
import os

print(os.getcwd())
```

Sys Module

```
import sys

print(sys.version)
```

What is a Package?

A **package** is a collection of related modules organized in directories.

In simple terms:

- Module = Single Python file
 - Package = Folder containing multiple modules
-

Package Structure

```
mypackage/  
|  
|— __init__.py  
|— arithmetic.py  
|— geometry.py
```

arithmetic.py

```
def add(a, b):  
    return a + b
```

geometry.py

```
def area_square(side):  
    return side * side
```

Using a Package

```
from mypackage.arithmetic import add  
from mypackage.geometry import area_square
```

```
print(add(5, 3))  
print(area_square(4))
```

Output:

```
8  
16
```

Role of init.py

`__init__.py` indicates that a directory should be treated as a Python package.

Example:

```
mypackage/  
|  
|— __init__.py  
|— module1.py  
|— module2.py
```

The file may be empty or contain initialization code.

Example:

```
print("Package initialized")
```

Difference Between Module and Package

Module	Package
Single Python file	Collection of modules
Ends with .py	Directory containing modules
Easier to create	Used for larger projects
Example: math.py	Example: numpy package

Popular Python Packages

Package	Purpose
NumPy	Numerical Computing
Pandas	Data Analysis
Matplotlib	Data Visualization
Seaborn	Statistical Visualization
Scikit-learn	Machine Learning
TensorFlow	Deep Learning
Flask	Web Development
Django	Web Development
OpenCV	Image Processing

Installing Packages

Python uses **pip** (Package Installer for Python).

Install a Package

`pip install numpy`

Install Pandas

`pip install pandas`

Install Matplotlib

`pip install matplotlib`

Viewing Installed Packages

`pip list`

Package Information

`pip show numpy`

Upgrading a Package

`pip install --upgrade numpy`

Uninstalling a Package

`pip uninstall numpy`

Error Types in Python

Identifying the specific exception type in Python is a crucial skill for debugging. Here are 15 problems where you need to determine exactly which **built-in error type** would be raised.

The Error Identification Challenge

#	Code Snippet	Target Error Type
1	<code>print(10 / 0)</code>	ZeroDivisionError
2	<code>int("apple")</code>	ValueError
3	<code>print(undefined_variable)</code>	NameError
4	<code>items = [1, 2, 3] print(items[5])</code>	IndexError
5	<code>info = {"name": "Alice"} print(info["age"])</code>	KeyError
6	<code>"hello" + 5</code>	TypeError
7	<code>import non_existent_module</code>	ModuleNotFoundError
8	<code>def func(): print("Missing Indent")</code>	IndentationError
9	<code>while True print("Hello")</code>	SyntaxError

10	<code>open("imaginary_file.txt", "r")</code>	FileNotFoundError
11	<code>x = [1, 2]; x.append(3, 4)</code>	TypeError
12	<code>from math import non_existent_function</code>	ImportError
13	<code>res = 2.0**1000000</code>	OverflowError
14	<code>my_list = [1]; my_list.remove(10)</code>	ValueError
15	<code>(5).upper()</code>	AttributeError

Key Distinctions to Remember

Understanding why an error occurs is just as important as knowing its name. Here is a quick breakdown of the "look-alikes":

- **ValueError vs. TypeError:**
 - **TypeError** happens when you use the wrong *type* (e.g., trying to add a string to an integer).
 - **ValueError** happens when the type is correct, but the *content* is inappropriate (e.g., `int()` accepts strings, but "apple" cannot be converted).
- **SyntaxError vs. IndentationError:**
 - **IndentationError** is a specific subclass of `SyntaxError` that only triggers when your spaces/tabs are inconsistent.
- **IndexError vs. KeyError:**
 - **IndexError** is for sequences (lists, tuples) using an integer position.
 - **KeyError** is for mappings (dictionaries) using a specific key.

OOPS in Python

Topic 1: Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organises code around objects — self-contained units combining data (attributes) and behaviour (methods). Python is fully object-oriented: everything in Python, including integers, strings, and functions, is an object.

1.1 What is OOP?

Instead of writing procedures that operate on data, OOP models real-world entities as objects. A bank account, a dog, a web page — each can be an object with properties and actions.

1.2 OOP vs Procedural Programming

Aspect	Procedural	Object-Oriented
Focus	Functions and procedures	Objects and classes
Data handling	Passed between functions	Encapsulated in objects
Code reuse	Function reuse	Inheritance and composition
Scalability	Harder for large programs	Designed for large systems
Real-world mapping	Indirect	Natural and intuitive

1.3 The Four Pillars of OOP

- **Encapsulation:** Bundling data and methods, hiding internal details.
- **Inheritance:** Child class acquires attributes and methods from parent.
- **Polymorphism:** Same interface, different behaviour for different types.
- **Abstraction:** Hiding complex details, exposing only the necessary interface.

1.4 Why Use OOP in Python?

- Modular, reusable, and maintainable code
- Natural modeling of real-world entities
- Foundation of Django, Flask, PyGame, and the standard library
- Everything in Python is already an object (int, str, list, function)

Key Insight

In Python, every value is an object. `x = 5` creates an instance of the `int` class. Verify with `type(x)` or `isinstance(x, int)`.

Topic 2: Classes & Objects

A class is a blueprint. An object (instance) is a specific realisation of that blueprint. The class defines what attributes and methods objects will have.

2.1 Defining a Class

```
# Syntax: class ClassName:
class Dog:
    pass    # 'pass' used when body is empty

# Naming convention: PascalCase (UpperCamelCase)
class BankAccount: pass
class StudentRecord: pass
```

2.2 Creating Objects (Instantiation)

```
class Dog:
    pass

# Creating objects
dog1 = Dog()    # dog1 is an instance of Dog
dog2 = Dog()    # dog2 is a DIFFERENT instance

print(dog1 is dog2)        # False - different objects in memory
print(type(dog1))          # <class '__main__.Dog'>
print(isinstance(dog1, Dog)) # True
print(id(dog1))            # Memory address
```

2.3 Class vs Instance Terminology

Term	Description	Example
Class	Blueprint / template	class Dog:
Object / Instance	Specific realisation	dog1 = Dog()
Instantiation	Act of creating an object	Dog()
Attribute	Data stored in an object	dog1.name
Method	Function inside a class	def bark(self):

2.4 A Complete Class Example

```
class Dog:
    species = 'Canis lupus familiaris' # Class attribute (shared)

    def __init__(self, name, breed, age):
        self.name = name # Instance attribute
        self.breed = breed
        self.age = age

    def bark(self):
        print(f'{self.name} says: Woof!')

    def info(self):
        return f'{self.name} is a {self.age}-year-old {self.breed}'

dog1 = Dog('Buddy', 'Labrador', 3)
dog2 = Dog('Max', 'Poodle', 5)
dog1.bark() # Buddy says: Woof!
print(dog2.info()) # Max is a 5-year-old Poodle
print(Dog.species) # Canis lupus familiaris
```

Topic 3: Attributes & Methods

3.1 Understanding 'self'

`self` is a reference to the current instance. It is the first parameter of every instance method. When you call `dog1.bark()`, Python automatically passes `dog1` as `self`.

How self works

`dog1.bark()` is equivalent to `Dog.bark(dog1)`. Python translates instance calls automatically. The name `'self'` is convention, not a keyword.

3.2 Class vs Instance Attributes

```
class Student:
    school_name = 'Python Academy' # Class attr (shared)
    total_students = 0              # Class attr

    def __init__(self, name, grade):
        self.name = name           # Instance attr (unique per object)
        self.grade = grade
        Student.total_students += 1

s1 = Student('Alice', 'A')
s2 = Student('Bob', 'B')
print(s1.school_name)             # Python Academy
print(Student.total_students)     # 2

# Overriding a class attribute on one instance
s1.school_name = 'New School'
print(s1.school_name)             # New School (only s1)
print(s2.school_name)             # Python Academy (unchanged)
```

3.3 Types of Methods

Method Type	First Param	Decorator	Access To
Instance Method	<code>self</code>	None	Instance + Class data
Class Method	<code>cls</code>	<code>@classmethod</code>	Class data only
Static Method	None	<code>@staticmethod</code>	Neither (pure utility)

```
class MathHelper:
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius

    # Instance method - can access self and class data
    def area(self):
```



```

        return MathHelper.pi * self.radius ** 2

    # Class method - can modify class-level data
    @classmethod
    def update_pi(cls, new_pi):
        cls.pi = new_pi

    # Static method - utility, no access to self or cls
    @staticmethod
    def circle_info():
        print('Area of circle = pi * r squared')

c = MathHelper(5)
print(c.area())           # 78.53975
MathHelper.update_pi(3.14159265)
MathHelper.circle_info()  # works without an instance

```

3.4 Dynamic Attribute Management

```

class Person:
    def __init__(self, name): self.name = name

p = Person('Alice')

print(getattr(p, 'name'))           # Alice
print(getattr(p, 'age', 'Unknown')) # Unknown (default)

setattr(p, 'age', 30) # dynamically add attribute
print(p.age)          # 30

print(hasattr(p, 'name')) # True
print(hasattr(p, 'email')) # False

delattr(p, 'age') # remove attribute
print(p.__dict__) # {'name': 'Alice'}

```

Topic 4: Constructors & Destructors

4.1 `__init__` — Initialiser Constructor

`__init__` is automatically called when an object is created. It sets up the object's initial state. It supports default arguments and keyword arguments.

```
class BankAccount:
    def __init__(self, owner, balance=0, currency='INR'):
        self.owner      = owner
        self.balance     = balance
        self.currency    = currency
        self.transactions = [] # mutable default inside __init__

    def deposit(self, amount):
        self.balance += amount
        self.transactions.append(('deposit', amount))
        return self # enables method chaining

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError('Insufficient funds')
        self.balance -= amount
        return self

    def statement(self):
        print(f'Owner: {self.owner}, Balance: {self.balance}')

acc = BankAccount('Arjun')
acc.deposit(2000).deposit(500).withdraw(200) # method chaining
acc.statement() # Owner: Arjun, Balance: 2300
```

4.2 `__new__` — Object Creator

`__new__` is the true constructor — it creates and returns the new object. `__init__` then initialises it. Normally you never override `__new__`, but it enables patterns like Singleton.

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()
print(s1 is s2) # True - only one instance ever exists

# Execution order:
# 1. __new__(cls) => creates and returns the object
# 2. __init__(self) => initialises the object
```

4.3 __del__ — Destructor

```
class FileHandler:
    def __init__(self, filename):
        self.filename = filename
        print(f'Opened {filename}')

    def __del__(self):
        print(f'Closed {self.filename}')

f = FileHandler('data.txt')  # Opened data.txt
del f                       # Closed data.txt
```

Warning

Do NOT rely on `__del__` for critical cleanup. Python's garbage collector does not guarantee when it runs. Use context managers (with statement) instead.

4.4 Context Managers — Best Practice for Cleanup

```
class DBConnection:
    def __init__(self, db_name):
        self.db_name = db_name

    def __enter__(self):
        print(f'Connecting to {self.db_name}...')
        return self  # assigned to 'as' variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        print('Closing connection.')
        return False  # False = don't suppress exceptions

    def query(self, sql):
        print(f'Running: {sql}')

with DBConnection('mydb') as db:
    db.query('SELECT * FROM users')
# Connection auto-closes even if an exception occurs
```

Topic 5: Encapsulation

Encapsulation bundles data and methods, controlling access to internals. Python uses naming conventions rather than strict keywords like private/protected.

5.1 Access Levels

Convention	Syntax	Meaning	Accessible From
Public	name	Fully accessible	Anywhere
Protected	<code>_name</code>	Internal use (convention)	Class and subclasses
Private	<code>__name</code>	Name mangling applied	Only within the class

5.2 Public, Protected and Private in Practice

```
class Employee:
    def __init__(self, name, dept, salary):
        self.name = name      # Public
        self._dept = dept    # Protected
        self.__salary = salary # Private (mangled to _Employee__salary)

    def get_info(self):
        return f'{self.name} | {self._dept} | Rs.{self.__salary}'

emp = Employee('Aryan', 'Engineering', 90000)
print(emp.name)           # OK
print(emp._dept)          # Works (not recommended)
# print(emp.__salary)     -> AttributeError!
print(emp._Employee__salary) # 90000 (name mangling access)
print(emp.get_info())      # Proper way
```

5.3 @property — Pythonic Getters and Setters

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):      # Getter
        return self._celsius

    @celsius.setter
    def celsius(self, value): # Setter with validation
        if value < -273.15:
            raise ValueError('Below absolute zero!')
        self._celsius = value

    @property
    def fahrenheit(self):  # Computed property
```

```
        return self._celsius * 9/5 + 32

    @celsius.deleter
    def celsius(self):    # Deleter
        del self._celsius

t = Temperature(25)
print(t.celsius)        # 25
print(t.fahrenheit)     # 77.0
t.celsius = 100         # Setter validates
# t.celsius = -300      -> ValueError!
```

Topic 6: Inheritance

Inheritance allows a child class (subclass) to acquire attributes and methods from a parent class (superclass). It establishes IS-A relationships and enables code reuse.

6.1 Types of Inheritance

Type	Description	Syntax
Single	One child from one parent	class B(A):
Multiple	Child from multiple parents	class C(A, B):
Multi-level	Chain: C inherits B which inherits A	class C(B): # B inherits A
Hierarchical	Multiple children from one parent	class B(A): and class C(A):
Hybrid	Combination of above types	Mix of all patterns

6.2 Single Inheritance with super()

```
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        return f'{self.name} says {self.sound}'

    def breathe(self):
        return 'Inhale... Exhale...'

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, 'Woof') # Call parent __init__
        self.breed = breed

    def fetch(self):
        return f'{self.name} fetches the ball!'

    def speak(self):
        # Method Overriding
        base = super().speak() # Call parent method
        return f'{base} (tail wagging)'

d = Dog('Rex', 'German Shepherd')
print(d.speak()) # Rex says Woof (tail wagging)
print(d.breathe()) # Inherited from Animal
print(isinstance(d, Animal)) # True
print(issubclass(Dog, Animal)) # True
```

6.3 Multiple Inheritance and Method Resolution Order (MRO)

```

class Flyable:
    def move(self): return 'Flying!'

class Swimmable:
    def move(self): return 'Swimming!'

class Duck(Flyable, Swimmable): # Multiple inheritance
    pass

d = Duck()
print(d.move()) # 'Flying!' - Flyable listed first
print(Duck.__mro__) # (Duck, Flyable, Swimmable, object)

# Diamond Problem - Python C3 Linearisation resolves it
class A:
    def hello(self): return 'A'
class B(A):
    def hello(self): return 'B'
class C(A):
    def hello(self): return 'C'
class D(B, C): pass

print(D().hello()) # 'B' - MRO: D -> B -> C -> A -> object

```

6.4 Mixins

Mixins are small classes that add reusable behaviour to other classes without deep inheritance chains. They are listed before the main parent class.

```

class LogMixin:
    def log(self, msg):
        print(f'[LOG] {self.__class__.__name__}: {msg}')

class JSONMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class User(LogMixin, JSONMixin):
    def __init__(self, name, email):
        self.name = name
        self.email = email

u = User('Priya', 'priya@example.com')
u.log('User created') # [LOG] User: User created
print(u.to_json()) # {"name": "Priya", "email": "..."}

```

Topic 7: Polymorphism

Polymorphism means 'many forms.' Different objects respond to the same method call differently. Python achieves this via duck typing — if it has the right methods, it works.

7.1 Method Overriding

```
class Shape:
    def area(self):
        raise NotImplementedError('Subclasses must implement area()')

    def describe(self):
        return f'I am {self.__class__.__name__} with area {self.area():.2f}'

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h

class Triangle(Shape):
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self): return 0.5 * self.b * self.h

shapes = [Circle(7), Rectangle(4, 6), Triangle(3, 8)]
for shape in shapes:
    print(shape.describe()) # Each calls its own area()
```

7.2 Duck Typing

```
class Cat:
    def speak(self): return 'Meow'

class Dog:
    def speak(self): return 'Woof'

class Robot:
    def speak(self): return 'Beep Boop' # Not an animal at all

def make_noise(entity): # Doesn't check type — only needs speak()
    print(entity.speak())

for thing in [Cat(), Dog(), Robot()]:
    make_noise(thing) # Meow / Woof / Beep Boop
```


7.3 Operator Overloading

```
class Vector:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, other): return Vector(self.x+other.x, self.y+other.y)
    def __sub__(self, other): return Vector(self.x-other.x, self.y-other.y)
    def __mul__(self, scalar): return Vector(self.x*scalar, self.y*scalar)
    def __eq__(self, other): return self.x==other.x and self.y==other.y
    def __repr__(self): return f'Vector({self.x}, {self.y})'
    def __abs__(self): return (self.x**2 + self.y**2) ** 0.5

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)    # Vector(4, 6)
print(v1 * 3)     # Vector(3, 6)
print(v1 == v2)   # False
print(abs(v2))    # 5.0
```

7.4 Operator Overloading Quick Reference

Method	Operator	Method	Operator
<code>__add__</code>	<code>a + b</code>	<code>__eq__</code>	<code>a == b</code>
<code>__sub__</code>	<code>a - b</code>	<code>__ne__</code>	<code>a != b</code>
<code>__mul__</code>	<code>a * b</code>	<code>__lt__</code>	<code>a < b</code>
<code>__truediv__</code>	<code>a / b</code>	<code>__le__</code>	<code>a <= b</code>
<code>__floordiv__</code>	<code>a // b</code>	<code>__gt__</code>	<code>a > b</code>
<code>__mod__</code>	<code>a % b</code>	<code>__ge__</code>	<code>a >= b</code>
<code>__pow__</code>	<code>a ** b</code>	<code>__neg__</code>	<code>-a</code>
<code>__len__</code>	<code>len(a)</code>	<code>__bool__</code>	<code>bool(a)</code>
<code>__call__</code>	<code>obj()</code>	<code>__hash__</code>	<code>hash(a)</code>

Topic 8: Abstraction

Abstraction hides internal complexity and exposes only a clean interface. In Python, use the `abc` (Abstract Base Classes) module. Abstract classes cannot be instantiated — they define contracts.

8.1 Abstract Base Classes (ABC)

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC): # Cannot be instantiated directly

    @abstractmethod
    def process_payment(self, amount):
        pass

    @abstractmethod
    def refund(self, transaction_id):
        pass

    def validate_amount(self, amount): # Concrete method
        if amount <= 0:
            raise ValueError('Amount must be positive')

class UPIGateway(PaymentGateway):
    def process_payment(self, amount):
        self.validate_amount(amount)
        print(f'UPI: Processing Rs.{amount}')
    def refund(self, txn_id):
        print(f'UPI: Refunding TXN {txn_id}')

class CardGateway(PaymentGateway):
    def process_payment(self, amount):
        self.validate_amount(amount)
        print(f'Card: Processing Rs.{amount}')
    def refund(self, txn_id):
        print(f'Card: Refunding TXN {txn_id}')

# PaymentGateway() -> TypeError! Cannot instantiate abstract class
gw = UPIGateway()
gw.process_payment(500)
```

8.2 Abstract Properties

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @property
    @abstractmethod
    def fuel_type(self): pass # Must implement as @property
```

```

    @abstractmethod
    def max_speed(self): pass

    def describe(self):
        return f'{self.__class__.__name__}: {self.fuel_type}, {self.max_speed()}
kmh'

class ElectricCar(Vehicle):
    @property
    def fuel_type(self): return 'Electric'
    def max_speed(self): return 250

class PetrolCar(Vehicle):
    @property
    def fuel_type(self): return 'Petrol'
    def max_speed(self): return 200

for car in [ElectricCar(), PetrolCar()]:
    print(car.describe())

```

Abstraction vs Encapsulation

Encapsulation: HIDING DATA (private attributes, getters/setters). Abstraction: HIDING COMPLEXITY (abstract classes, interfaces). They work together.

Topic 9: Magic / Dunder Methods

Double-underscore (dunder) methods are special methods Python calls automatically in response to built-in operations. They make your objects behave like built-in types.

9.1 String Representation

```
class Book:
    def __init__(self, title, author, price):
        self.title, self.author, self.price = title, author, price

    def __str__(self):    # User-facing: print(obj), str(obj)
        return f'"{self.title}" by {self.author}'

    def __repr__(self):  # Dev-facing: repr(obj), REPL display
        return f'Book("{self.title}", "{self.author}", {self.price})'

b = Book('Python OOP', 'Guido', 499)
print(b)           # "Python OOP" by Guido
print(repr(b))     # Book("Python OOP", "Guido", 499)
```

9.2 Container Protocol

```
class Library:
    def __init__(self): self._books = []
    def add(self, book): self._books.append(book)

    def __len__(self):      return len(self._books)      # len(lib)
    def __getitem__(self, idx): return self._books[idx]   # lib[0]
    def __setitem__(self, i, v): self._books[i] = v       # lib[0] = x
    def __delitem__(self, idx): del self._books[idx]      # del lib[0]
    def __contains__(self, b): return b in self._books    # x in lib
    def __iter__(self):     return iter(self._books)      # for x in lib
    def __reversed__(self): return reversed(self._books)

lib = Library()
lib.add('Clean Code')
lib.add('Python Tricks')
print(len(lib))           # 2
print(lib[0])             # Clean Code
print('Clean Code' in lib) # True
for book in lib: print(book)
```

9.3 Numeric Dunder Methods

```
class Money:
    def __init__(self, amount, currency='INR'):
        self.amount = amount
        self.currency = currency

    def __add__(self, other): return Money(self.amount + other.amount)
```

```

def __sub__(self, other): return Money(self.amount - other.amount)
def __mul__(self, factor): return Money(self.amount * factor)
def __eq__(self, other): return self.amount == other.amount
def __lt__(self, other): return self.amount < other.amount
def __le__(self, other): return self.amount <= other.amount
def __bool__(self): return self.amount > 0
def __repr__(self): return f'Money({self.amount} {self.currency})'

```

```

a = Money(500)
b = Money(300)
print(a + b)    # Money(800 INR)
print(a > b)    # True
print(bool(Money(0))) # False

```

9.4 __call__ — Callable Objects

```

class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value): # Makes instance callable like a function
        return value * self.factor

triple = Multiplier(3)
print(triple(5))           # 15
print(triple(10))          # 30
print(callable(triple))    # True

```

9.5 Complete Dunder Method Reference

Method	Trigger	Method	Trigger
<code>__init__</code>	<code>obj = Class()</code>	<code>__del__</code>	<code>del obj</code> / GC
<code>__new__</code>	Object creation	<code>__str__</code>	<code>str(obj)</code>
<code>__repr__</code>	<code>repr(obj)</code>	<code>__len__</code>	<code>len(obj)</code>
<code>__getitem__</code>	<code>obj[key]</code>	<code>__setitem__</code>	<code>obj[k]=v</code>
<code>__contains__</code>	<code>x in obj</code>	<code>__iter__</code>	<code>for x in obj</code>
<code>__enter__</code>	<code>with obj as</code>	<code>__exit__</code>	end of <code>with</code>
<code>__add__</code>	<code>a + b</code>	<code>__sub__</code>	<code>a - b</code>
<code>__mul__</code>	<code>a * b</code>	<code>__truediv__</code>	<code>a / b</code>
<code>__eq__</code>	<code>a == b</code>	<code>__lt__</code>	<code>a < b</code>
<code>__bool__</code>	<code>bool(a)</code>	<code>__call__</code>	<code>obj()</code>
<code>__hash__</code>	<code>hash(a)</code>	<code>__abs__</code>	<code>abs(a)</code>